Spec-Driven Development

From Vibe Coding to Structured AI Execution

A comprehensive guide to spec-driven development (SDD) methodologies and tools for AI coding agents, including GitHub SpecKit, AWS Kiro, AI-DLC Workflows, OpenSpec, and VisionSpec.

5 Levels v1.0 Last updated June 2026

spec-driven sdd agents specifications ai-native methodology

In February 2025, Andrej Karpathy coined the term “vibe coding”—describing software development where you prompt an AI with rough intentions and it generates code. By early 2026, vibe coding had become mainstream: 92% of US developers use AI tools daily, and 41% of global code is AI-generated. Tools like **Cursor** grew to over 1 million daily active users on this paradigm.

But something went wrong.

A December 2025 CodeRabbit analysis found that AI co-authored code contained 1.7x more major issues compared to human-written code, with security vulnerabilities 2.74x higher. In early 2026, a vibe-coded app suffered a major data breach exposing 1.5 million API keys. Engineers began warning of a “vibe slop” crisis—companies trading near-term productivity for longer-term problems.

Even Karpathy, who coined the term, acknowledged the shift: “This era is ending and we are entering the age of agentic engineering—orchestrating agents against detailed specifications with human oversight.”

This article introduces **Spec-Driven Development (SDD)**—the methodology replacing vibe coding for production software. We examine the core concepts, compare five

major tools (GitHub SpecKit, AWS Kiro, AI-DLC Workflows, OpenSpec, and VisionSpec), and show how SDD connects to the **Software Delivery Autonomy Levels** and **Loop Engineering** frameworks.

What Is Spec-Driven Development?

Spec-Driven Development (SDD) is a software development methodology where an executable, version-controlled specification—not the code—is the single source of truth.

The team (or an AI coding agent) first writes a detailed spec describing what the system should do, then derives an implementation plan, breaks it into atomic tasks, and only then generates the code.

The SDD Inversion

Traditional development treats specifications as secondary guidance documents that quickly become outdated. SDD inverts this hierarchy:

Traditional Development	Spec-Driven Development
Code is primary	Specification is primary
Specs describe code	Code implements specs
Requirements drift from implementation	Requirements drive implementation
Documentation is afterthought	Documentation is executable
AI generates from prompts	AI generates from specifications

Why SDD Matters Now

SDD emerged as a direct response to the failure modes of vibe coding:

Problem 1: Agents drift from intent. Without explicit scope, agents make assumptions and head in the wrong direction fast. A spec anchors the agent to defined behavior.

Problem 2: No traceability. When requirements live only in chat history, there's no way to verify that implementation matches intent. Specs create an auditable trail.

Problem 3: AI "games" narrow tests. Traditional unit tests can be gamed with hardcoded return values. SDD uses acceptance criteria and scenarios that describe observable behavior, not implementation details.

Problem 4: Brownfield complexity. Most development happens in existing codebases. SDD tools use delta-based specifications that describe only what's changing, not the entire system.

Early adopter reports from GitHub and AWS show **3-10x higher first-pass success rate** from AI agents on non-trivial tasks when using SDD versus vibe coding.

Core SDD Concepts

All SDD tools share common patterns, though implementations vary.

1. Specification as Source of Truth

Specifications define **what** and **why**, not **how**. They use requirements language (SHALL, MUST, SHOULD, MAY) and acceptance criteria in Given/When/Then format:

```
### Requirement: Session Expiration
The system MUST expire sessions after 30 minutes of inactivity.

#### Scenario: Idle timeout
- GIVEN an authenticated session
- WHEN 30 minutes pass without activity
- THEN the session is invalidated
```

2. Multi-Phase Workflow

SDD workflows separate concerns into phases:

Phase	Purpose	Artifacts
Specification	Define what to build	Requirements, user stories, scenarios
Design	Define how to build	Architecture, data models, APIs
Planning	Define tasks	Implementation checklist
Execution	Build it	Code, tests, documentation
Verification	Confirm correctness	Acceptance tests, quality gates

3. Structured Artifacts

SDD tools generate standardized artifacts rather than freeform documents:

PRD (Product Requirements Document) — What users need
TRD (Technical Requirements Document) — How to implement
Acceptance Criteria — Observable behavior to verify
Task Lists — Atomic implementation steps with checkboxes
Design Documents — Architecture decisions and rationales

4. Quality Gates

Each phase includes evaluation and approval:

Rubric-based evaluation — AI judges score specs against criteria
Conflict detection — Automatic identification of contradictions
Human approval — Explicit sign-off before proceeding
Constitution compliance — Enforcement of architectural principles

5. Delta-Based Changes

For brownfield development, SDD tools describe only what's changing:

```
## ADDED Requirements
### Session Timeout Configuration
The system SHALL allow administrators to configure session timeout.

## MODIFIED Requirements
### Session Expiration
The system MUST expire sessions after the configured timeout (default: 30

## REMOVED Requirements
### Hard-coded Timeout
(Replaced by configurable timeout above)
```

This enables specification of changes to large codebases without documenting the entire system first.

SDD Tool Landscape

Five major tools have emerged for spec-driven development with AI coding agents. Each takes a different approach while sharing core SDD principles.

GitHub SpecKit

Philosophy: Constitution-governed, multi-agent orchestration with extensive customization.

Key Innovation: A "Constitution" layer defines architectural principles (Articles I-IX) that constrain all generated implementations. Templates act as sophisticated prompts that prevent premature implementation and enforce test-first thinking.

Structure:

```
.specify/
├── memory/constitution.md    # Architectural principles
├── templates/                # Spec templates
├── extensions/               # Added capabilities
└── presets/                  # Organizational customizations

specs/
└── 001-feature-name/
    ├── spec.md               # Requirements (WHAT + WHY)
    ├── plan.md               # Technical design (HOW)
    ├── tasks.md              # Implementation checklist
    └── research.md           # Technical investigation
```

Workflow Commands:

```
/speckit.constitution  → Establish project principles
/speckit.specify       → Define feature requirements
/speckit.clarify       → Resolve ambiguities
/speckit.plan          → Create technical design
/speckit.tasks         → Generate task breakdown
/speckit.implement     → Execute tasks
/speckit.converge      → Verify completion
```

Integrations: 30+ AI agents including Claude Code, Cursor, Copilot, Codex, Gemini CLI, Windsurf, and Kiro.

Best For: Teams wanting maximum governance and customization. Enterprise organizations with established architectural standards.

AWS Kiro

Philosophy: Structured specs with EARS notation and three workflow types (feature, bugfix, quick plan).

Key Innovation: Uses EARS (Easy Approach to Requirements Syntax) notation: "WHEN [condition/event] THE SYSTEM SHALL [expected behavior]". Bugfix specs explicitly capture unchanged behavior for regression prevention.

Feature Spec Structure:

```
.kiro/specs/feature-name/
├── requirements.md    # EARS-notation requirements
├── design.md          # Technical architecture
└── tasks/             # Implementation tasks
```

Bugfix Spec Structure:

```
## Current Behavior (Defect)
WHEN [condition] THEN the system [incorrect behavior]

## Expected Behavior (Correct)
WHEN [condition] THEN the system SHALL [correct behavior]

## Unchanged Behavior (Regression Prevention)
WHEN [condition] THEN the system SHALL CONTINUE TO [existing behavior]
```

Workflow Variants:

- **Requirements-First** — Start with behavior, derive design

Design-First — Start with architecture, derive requirements
Quick Plan — Auto-generate all phases without approval gates

Best For: Teams wanting structured requirements without extensive setup. Projects with strict compliance or audit requirements.

AWS AI-DLC Workflows

Philosophy: Adaptive three-phase methodology that executes only stages that add value.

Key Innovation: Stages are conditional—the workflow adapts to project type (greenfield vs brownfield), complexity level (minimal/standard/comprehensive), and work type (feature vs bug fix). Includes reverse engineering phase for existing codebases.

Three-Phase Structure:

Phase 1 — INCEPTION (Planning):

Workspace Detection (always)
Reverse Engineering (brownfield only)
Requirements Analysis (adaptive depth)
User Stories (user-facing changes)
Application Design (complex changes)
Units Generation (system decomposition)

Phase 2 — CONSTRUCTION (Implementation):

Functional Design (per unit)
NFR Requirements (performance, security)
Infrastructure Design (cloud mapping)
Code Generation (planning + execution)

Build and Test (comprehensive testing)

Phase 3 — OPERATIONS (Deployment):

Deployment automation
Monitoring and observability
Incident response

Artifact Structure:

```
aidlc-docs/
├── aidlc-state.md           # Workflow progress
├── audit.md                 # Complete audit trail
├── inception/
│   ├── requirements/
│   ├── user-stories/
│   └── application-design/
└── construction/
    ├── {unit-name}/
    │   ├── functional-design/
    │   └── code/
    └── build-and-test/
```

Question-File Pattern: Questions are never asked inline. AI creates question files with answer tags that users fill in, ensuring durable records for team collaboration.

Best For: Large enterprise projects. AWS-native development. Teams wanting comprehensive lifecycle coverage.

OpenSpec

Philosophy: Fluid, brownfield-first workflow with delta-based specifications.

Key Innovation: Separates `specs/` (current system behavior) from `changes/` (proposed modifications). Delta specs describe only ADDED/MODIFIED/REMOVED requirements, making it practical to specify changes to large existing codebases.

Structure:

```
openspec/
├── specs/                    # Source of truth (current behavior)
│   ├── auth/spec.md
│   ├── payments/spec.md
│   └── ui/spec.md
├── changes/                  # Proposed modifications
│   └── add-dark-mode/
│       ├── proposal.md       # Why this change
│       ├── design.md         # Technical approach
│       ├── tasks.md          # Implementation checklist
│       └── specs/            # Delta specs (changes only)
│           └── ui/spec.md
└── config.yaml               # Project configuration
```

Delta Spec Format:

```
## ADDED Requirements
### Dark Mode Toggle
The system SHALL provide a toggle in user settings.

## MODIFIED Requirements
### Theme Application
The system SHALL apply the selected theme to all UI components.
(Previously: fixed light theme)
```

Workflow Commands:

```
/opsx:explore   → Think through options (no artifacts)
/opsx:propose   → Create change with all artifacts
/opsx:apply     → Implement tasks
/opsx:archive   → Merge delta specs into main specs
```

Archive Behavior: When a change is archived, delta specs are merged into main specs—ADDED requirements append, MODIFIED requirements replace, REMOVED requirements delete.

Integrations: 25+ AI tools including Claude Code, Cursor, Windsurf, Copilot, Cline, and Continue.

Best For: Brownfield development. Teams wanting lightweight workflow. Projects where existing codebase documentation is sparse.

VisionSpec

Philosophy: Multi-domain orchestration implementing Amazon's Working Backwards methodology.

Key Innovation: Bridges organizational strategy with AI execution through a structured flow: Market Requirements → Press Release (vision) → FAQ (scope) → Product Requirements → Technical Requirements → Unified Execution Spec. Includes post-ship alignment to keep specs synchronized with reality.

Working Backwards Flow:

```
MRD (Market)
    ↓
Press Release (vision)  →  FAQ (scope challenges)
    ↓
PRD (Product Requirements)
    ↓
UXD (User Experience)
    ↓
TRD (Technical)  →  TPD (Test Plan)  →  IRD (Infrastructure)
    ↓
spec.md (Unified Execution Spec)
    ↓
Export to: SpecKit | AI-DLC | OpenSpec | GitHub Issues | Jira
    ↓
current-truth.md (Post-Ship Alignment)
```

Directory Structure:

```
docs/specs/{project}/
├── source/                  # Human-authored specs
│   ├── mrd.md              # Market Requirements
│   ├── prd.md              # Product Requirements
│   └── uxd.md              # User Experience
├── gtm/                     # LLM-generated GTM docs
│   ├── press.md            # Press Release
│   ├── faq.md              # FAQ
│   └── narrative-6p.md     # AWS-style narrative
├── technical/               # LLM-generated technical
│   ├── trd.md              # Technical Requirements
│   ├── tpd.md              # Test Plan
│   └── ird.md              # Infrastructure
├── eval/                    # Evaluations (JSON)
├── spec.md                  # Unified execution spec
└── current-truth.md         # Post-ship reality
```

Methodology Profiles (12 total):

- **aws-product** — Working Backwards for new products

google — Design Docs + RFC (engineering-heavy)
stripe — API-First (platform products)
lean-startup — Build-Measure-Learn
design-thinking — Stanford d.school
jtbd — Jobs to be Done
continuous-discovery — Teresa Torres opportunity tree
shapeup — Basecamp 6-week cycles

MCP Server: First-class Model Context Protocol integration with 28+ tools for spec operations, synthesis, reconciliation, and export.

Export Targets: SpecKit, AI-DLC, OpenSpec, GitHub Issues, Jira, and custom formats.

Best For: Organizations implementing Working Backwards methodology. Multi-team coordination. Enterprises needing traceability from strategy to execution.

Tool Comparison Matrix

Capability	SpecKit	Kiro	AI-DLC	OpenSpec	Vis
Philosophy	Constitution-governed	EARS notation	Adaptive phases	Delta-based	Wo Ba
Primary Use	Greenfield features	Features + bugfixes	Full lifecycle	Brownfield changes	Str exe
Spec Format	Markdown templates	EARS requirements	Adaptive depth	Delta specs	Mu do
Artifact Count	4 core + extensions	3 per spec type	15+ per phase	4 per change	12 typ
Agent Integrations	30+	Kiro + Q Developer	7+ IDE/agents	25+	MC + s
Evaluation	Checklist-based	Implicit in workflow	Evaluator tool	Validation CLI	Ru bas
Customization	Extensions + presets	Settings	Rule details	Schemas	Pro Ch
Learning Curve	Medium-high	Low-medium	High	Low	Me
Best For	Governed enterprise	Structured requirements	AWS/enterprise	Brownfield	Str alig

Choosing an SDD Tool

Decision Framework

Start with VisionSpec if:

- You need alignment from business strategy to technical execution
- Multiple teams must coordinate on shared requirements
- You're implementing Working Backwards or similar methodology
- You want to export to multiple execution systems

Start with SpecKit if:

- You want maximum customization and governance
- You have established architectural principles to enforce
- You need integration with many different AI agents
- You're building a new codebase from scratch

Start with Kiro if:

- You want structured specs with minimal setup
- You need both feature and bugfix workflows
- You're already using AWS Q Developer
- You prefer EARS notation for requirements

Start with AI-DLC if:

- You're building on AWS infrastructure
- You need comprehensive lifecycle coverage
- You want adaptive depth based on complexity
- You have both greenfield and brownfield projects

Start with OpenSpec if:

- You're primarily doing brownfield development

You want a lightweight, fluid workflow
You don't need extensive governance
You want to track specs as source of truth

Combining Tools

These tools are not mutually exclusive. A common pattern:

VisionSpec for strategy-to-spec (MRD → PRD → TRD)
Export to SpecKit, AI-DLC, or OpenSpec
Execute with tool-specific workflows
Align post-ship with VisionSpec current-truth

VisionSpec's export targets include SpecKit, AI-DLC, and OpenSpec formats, enabling this orchestration.

SDD and the Autonomy Levels

Spec-Driven Development enables progression through the Software Delivery Autonomy Levels:

Level	SDD Role
Level 4: AI-Native Workflows	Specs replace ad-hoc prompts as AI input
Level 5: Agentic Engineering	Specs enable overnight autonomous sessions
Level 6: Autonomous Coding & Review	Specs + scenarios replace human code review
Level 7: Autonomous Operations	Specs define operational policies

At Level 5, the combination of SDD with **Loop Engineering** creates a powerful pattern:

```
REAL Loop (Mission-Driven):
  Read: spec.md + current implementation state
  Evaluate: progress against requirements
  Act: implement next requirement
  Loop: until all scenarios pass

VEAL Loop (Validation-Driven):
  Validate: run acceptance scenarios
  Evaluate: GO/NO-GO against spec
  Act: fix failing scenarios
  Loop: until all pass or escalate
```

The spec becomes the **mission** for REAL loops and the **validation criteria** for VEAL loops.

At Level 6, specs enable **scenario-based validation** that replaces human code review:

> "Of all the observed trajectories through all the scenarios, what fraction of them likely satisfy the user?"

This probabilistic framing—enabled by well-specified acceptance criteria—is what makes autonomous coding without human review possible.

Practical SDD Workflow

Here's a minimal SDD workflow that works with any tool:

1. Capture Intent (Not Implementation)

Start with what users need, not how to build it:

```
## User Story
As a team administrator, I want to set session timeout duration
so that I can balance security and convenience for my organization.

## Acceptance Criteria
- GIVEN I am a team administrator
- WHEN I navigate to security settings
- THEN I see a session timeout configuration option

- GIVEN I set timeout to 60 minutes
- WHEN a user's session exceeds 60 minutes of inactivity
- THEN the session is automatically terminated
```

2. Clarify Ambiguities

Before design, identify and resolve unclear requirements:

```
## Questions
1. What is the valid range for timeout values? [Answer: 5–480 minutes]
2. Should timeout apply to API sessions? [Answer: No, only web sessions]
3. What happens to unsaved work? [Answer: Show warning at 5 minutes befo
```

3. Design with Constraints

Technical design should reference requirements:

```
## Technical Design

### Implements: Session Timeout Configuration

**Approach:** Add `session_timeout_minutes` column to `team_settings` tab
Default value: 30 (matches current hard-coded behavior).

**Data Model:**
- teams.session_timeout_minutes: INTEGER, NOT NULL, DEFAULT 30
- Constraint: CHECK (session_timeout_minutes BETWEEN 5 AND 480)

**API Changes:**
- GET/PATCH /api/teams/{id}/settings — Add timeout field
```

4. Generate Tasks

Break design into atomic, testable tasks:

```
## Tasks

### Phase 1: Database
- [ ] Add migration for session_timeout_minutes column
- [ ] Add model validation for range constraint

### Phase 2: API
- [ ] Add timeout field to team settings serializer
- [ ] Add PATCH endpoint for updating timeout
- [ ] Add integration tests for timeout validation

### Phase 3: Frontend
- [ ] Add timeout input to security settings page
- [ ] Add client-side validation for range
- [ ] Add confirmation dialog for changes

### Phase 4: Session Management
- [ ] Update session expiration logic to use team setting
- [ ] Add warning notification at 5 minutes before expiry
- [ ] Add tests for custom timeout behavior
```

5. Execute with Tracking

Use AI agents to implement tasks while tracking progress:

```
/speckit.implement
or
/opsx:apply
or
claude "Implement the next unchecked task in tasks.md"
```

6. Verify Against Spec

Run acceptance scenarios, not just unit tests:

```
# Scenario: Custom timeout
Given: Team admin sets timeout to 60 minutes
When: User session inactive for 61 minutes
Then: Session terminated ✓

# Scenario: Warning notification
Given: Team admin sets timeout to 60 minutes
When: User session inactive for 55 minutes
Then: Warning notification displayed ✓
```

Anti-Patterns to Avoid

1. Spec as Afterthought

Wrong: Write code, then document what you built. **Right:** Write spec, get approval, then implement.

The spec is a contract, not documentation.

2. Implementation Details in Specs

Wrong: "Use React useState hook for form state management." **Right:** "Form SHALL preserve user input during validation errors."

Specs describe observable behavior, not implementation choices.

3. Skipping Clarification

Wrong: Make assumptions about unclear requirements. **Right:** Document questions and get explicit answers.

AI agents confidently implement wrong interpretations of ambiguous specs.

4. Monolithic Specs

Wrong: One massive spec file for the entire feature. **Right:** Modular specs organized by domain or capability.

Large specs exceed context windows and reduce coherence.

5. Ignoring Delta Specs for Brownfield

Wrong: Document entire system before making changes. **Right:** Use ADDED/MODIFIED/REMOVED format for changes.

Delta specs make brownfield development practical.

The Future of SDD

Spec-Driven Development is evolving rapidly:

Executable Specifications. Tools are moving toward specs that directly generate implementations—change a requirement, automatically regenerate affected code.

Continuous Spec-Reality Alignment. Post-ship alignment (pioneered by VisionSpec) will become standard—specs automatically updated to reflect production behavior.

Multi-Agent Orchestration. Specialized agents (requirements analyst, architect, developer, tester) will collaborate on specs, each contributing their expertise.

Specification as API. Specs will become machine-readable contracts that AI systems consume directly, not just guidance documents.

The trajectory is clear: **specifications become the primary artifact, and code becomes a generated output serving the specification.**

This is not a minor process improvement. It is a fundamental inversion of how software gets built.

Conclusion

Vibe coding was the first generation of AI-assisted development—powerful but unpredictable. Spec-Driven Development is the second generation: structured, auditable, and production-ready.

The five tools we examined represent different approaches to the same insight:

> AI agents are dramatically more effective when they operate against explicit, version-controlled specifications rather than freeform prompts.

GitHub SpecKit provides maximum governance through constitutions and templates. **AWS Kiro** offers structured EARS-notation requirements. **AI-DLC Workflows** delivers comprehensive lifecycle coverage. **OpenSpec** enables practical brownfield development through delta specs. **VisionSpec** bridges organizational strategy with AI execution.

The engineers who thrive in this environment will be those who master not just AI tooling, but the discipline of specification—defining what to build before building it.

As the Software Delivery Autonomy Levels progress from Level 4 (AI-Native Workflows) through Level 6 (Autonomous Coding & Review), SDD provides the foundation. Without structured specifications, autonomous coding is merely faster chaos. With them, it becomes reliable production software.

The market validates both approaches: Cursor scaled vibe coding to 1M+ daily users, while GitHub Copilot Workspace brought spec-driven workflows to 20M users. The tension between speed and structure is real—but for production software at scale, specifications win.

The vibe era is ending. The spec era has begun.

References

Thoughtworks. “Spec-driven development: Unpacking one of 2025’s key new AI-assisted engineering practices.” 2025.

GitHub. Spec Kit Documentation. github.com/github/spec-kit

AWS. Kiro Documentation. kiro.dev/docs/specs/

AWS Labs. AI-DLC Workflows. github.com/awslabs/aidlc-workflows

Fission AI. OpenSpec Documentation. github.com/Fission-AI/openspec

ProductBuildersHQ. VisionSpec. github.com/ProductBuildersHQ/visionspec

Karpathy, Andrej. “Vibe Coding.” February 2025.

CodeRabbit. “AI Code Quality Analysis.” December 2025.

DeepLearning.AI. “Spec-Driven Development with Coding Agents.” 2026.

ProductBuildersHQ. “Software Delivery Autonomy Levels.” productbuildershq.com/frameworks/software-delivery-autonomy

ProductBuildersHQ. “Loop Engineering.” productbuildershq.com/frameworks/loop-engineering